COURSE PLAN

A. COURSE OVERVIEW

Degree:	BE	Programme:	AI
Academic Year:	2025-26	Semester:	VI
Course Title:	Natural Language Processing	Course Code:	BAI601
L-T-P-S:	3-2-2-0	Duration of SEE	3 Hrs
Total Contact Hours:	40 Hrs (Theory) + 8-10 Lab slot	SEE Marks:	50*
CIE Marks:	50	IA Test	25
Credits:	4	Components	25
Lesson Plan Author:	Mr. Naveena C	Naveena	Date : 21/01/26
Checked By:	Mr. Venkatesh Y C		Date : 21/01/26

*The SEE will be conducted for 100 marks and proportionally reduced to 50 marks.

B. PREREQUISITES

- Mathematics for Computer Science (BCS301)
- Data Structures and Applications (BCS304)
- Python Programming for Data Science (BDS306B)
- Analysis & Design of Algorithms (BCS401)
- Theory of Computation (BCS503)

C. COURSE DESCRIPTION

i) Course Outcomes

At the end of the course, the student will be able to;

1. Apply the fundamental concept of NLP, grammar-based language model and statistical-based language model.
2. Explain morphological analysis and different parsing approaches.
3. Develop the Naive Bayes classifier and sentiment analysis for Natural language problems and text classifications.
4. Apply the concepts of information retrieval, lexical semantics, lexical dictionaries.
5. Identify the Machine Translation applications of NLP using Encode and Decoder.

ii) Relevance of the Course

- Machine Learning – 1 (BAI602)
- Introduction to Artificial Intelligence (BAI654D)
- Deep Learning & Reinforcement Learning (BAI701)
- Data Security & Privacy (BAD703)

iii) Applications areas

- Sentiment Analysis
- Machine Translation
- Speech Recognition
- Information Retrieval

Prepared by: Naveena C	Checked by: Venkatesh Y C	HOD

COURSE PLAN

D1. ARTICULATION MATRIX, CO v/s PO

Mapping of CO to PO												
	POs											
COs	1	2	3	4	5	6	7	8	9	10	11	12
1. Apply the fundamental concept of NLP, grammar-based language model and statistical-based language model.	3	2	2	-	-	-	-	-	-	-	-	-
2. Explain morphological analysis and different parsing approaches.	2	2	2	-	-	-	-	-	-	-	-	-
3. Develop the Naive Bayes classifier and sentiment analysis for Natural language problems and text classifications	2	2	3	-	-	-	-	-	-	-	-	-
4. Apply the concepts of information retrieval, lexical semantics, lexical dictionaries.	2	2	3	-	2	-	-	-	-	-	-	-
5. Identify the Machine Translation applications of NLP using Encode and Decoder.	2	2	3	-	3	-	-	-	-	-	-	-
Note: Mappings in the Tables D1 (above) and D2 (below) are done by entering in the corresponding cell the Correllation Levels in terms of numbers. For Slight (Low): 1, Moderate (Medium): 2, Substantial (High): 3 and for no correllation: " - ".												

D2. ARTICULATION MATRIX, CO v/s PSO

Mapping of CO to PSO		
	PSOs	
COs	1	2
1. Apply the fundamental concept of NLP, grammar-based language model and statistical-based language model.	-	3
2. Explain morphological analysis and different parsing approaches.	-	3
3. Develop the Naive Bayes classifier and sentiment analysis for Natural language problems and text classifications	-	3
4. Apply the concepts of information retrieval, lexical semantics, lexical dictionaries.	-	3
5. Identify the Machine Translation applications of NLP using Encode and Decoder.	-	3

E. MODULE PLANS

MODULE – I

Title:	Introduction and Language Modeling	Appr. Time:	8 Hrs
MO:			RBT
At the end of the Module, the student will be able to:			
1. Understand the basics of Natural Language Processing (NLP), its origins, and challenges and its applications.			L2
2. Apply statistical language models such as N-gram models (unigram, bigram).			L3
3. Understand Paninian framework and Karaka theory in linguistic analysis.			L2
Lesson Schedule:			
Lecture No.	Portion to be covered		CO
1	Introduction to Natural Language Processing		CO1
2	The origins of NLP, Language and Knowledge		CO1
3	The Challenges of NLP		CO1

COURSE PLAN

4	The Language and Grammar	CO1
5	The Processing Indian Languages	CO1
6	The NLP Applications	CO1
7	The Statistical Language Model - N-gram model (unigram, bigram)	CO1
8	The Paninion Framework, Karaka theory	CO1

Application Areas:

- Predictive text systems (e.g., autocomplete).
- Statistical modeling for speech recognition.
- Processing Indian languages for regional NLP applications

Review Questions / Questions Appeared in the Previous Years (CO):

1	What is NLP? List and explain applications of NLP. (CO1)
2	List and explain the challenges of NLP.(CO1)
3	Briefly explain various levels of natural language processing.(CO1)
4	Describe how to process Indian Languages. (CO1)
5	Explain in detail about Transformational grammar. (CO1)
6	Explain Statistical Language Model - N-gram model (unigram, bigram).(CO1)
7	Explain n-gram modelling of natural languages. Find the probability of the test sentence S4 in the following training set S1: The Arabian knights S2: These are the fairy tales of the east S3: The stories of the Arabian Knights are translated in many languages S4: The Arabian Knights are the fairy tales of the east (CO1)
8	Using Bi-gram model, find the probability of the sentence "*They play in a garden*" for the following training set data; *There is a big garden* *Children play in the garden* *They play inside beautiful garden* (CO1)
9	Explain the Paninion Framework for Indian Languages. (CO1)
10	Explain layered representation of Paninion Framework and Karaka theory.(CO1)

MODULE – II

Title:	Word Level Analysis and Syntactic Analysis	Appr. Time:	8 Hrs
MO:			RBT
At the end of the Module, the student will be able to:			
1. Implement word-level analysis using regular expressions and finite-state automata.			L3
2. Perform morphological parsing and spelling error detection & correction.			L2
3. Identify words, word classes, and apply part-of-speech tagging.			L1
4. Analyze sentence structure using context-free grammar and parsing techniques like topdown, bottom-up, and CYK parsing.			L3

Lesson Schedule:

Lecture No.	Portion to be covered	CO
1	Word Level Analysis: Regular Expressions and Finite-State Automata.	CO2
2	Morphological Parsing	CO2
3	Spelling Error Detection and Correction	CO2
4	Words and Word Classes, Part-of Speech Tagging	CO2
5	Syntactic Analysis: Context-Free Grammar	CO2

COURSE PLAN

6	Constituency	CO2
7	Top-down and Bottom-up	CO2
8	Parsing, CYK Parsing	CO2

Application Areas:

- Grammar-checking tools (e.g., Grammarly).
- Syntax-based query systems.
- Morphological analysis for low-resource languages.

Review Questions / Questions Appeared in the Previous Years (CO):

1	Explain two step morphological parser with a neat diagram.(CO2)
2	What are regular Expressions? Write regular expression for an email address. (CO2)
3	Discuss DFA and NFA. Mention the properties of finite automation.(CO2)
4	What is POS tagging? Explain Rule based taggers and Hybrid tagger.(CO2)
5	With example, explain basic top-down depth first algorithm.(CO2)
6	Illustrate minimum edit distance algorithm with example.(CO2)
7	How to detect spelling error and how to correct it? (CO2)
8	Explain Minimum edit distance algorithm and compute the minimum edit distance between tumour and tutor. (CO2)
9	Explain CYK algorithm.(CO2)
10	Discuss CYK parser. Tabulate the sequence of states created by CYK algorithm while parsing, a) The sun rises in the east b) The girl wrote an essay (CO2)

S->NP VP	S->VP	NP->Det Noun
NP->Noun	NP->NP PP	VP->VP NP
VP->Verb	VP->VP PP	PP->Preposition NP

MODULE – III

Title:	Naive Bayes, Text Classification and Sentiment	Appr. Time:	8 Hrs
MO:			RBT
At the end of the Module, the student will be able to:			
1. Understand and implement the Naive Bayes classification algorithm for text classification.			L2
2. Train and optimize Naive Bayes classifiers for sentiment analysis.			L3
3. Apply Naive Bayes in different NLP tasks, including text classification and language modeling.			L3

Lesson Schedule:

Lecture No.	Portion to be covered	CO
1	Naive Bayes Classifiers	CO3
2	Training the Naive Bayes Classifier	CO3
3	Training the Naive Bayes Classifier continued	CO3
4	Worked Example	CO3
5	Optimizing for Sentiment Analysis	CO3
6	Naive Bayes for Other Text Classification Tasks	CO3
7	Naive Bayes as a Language Model	CO3
8	Naive Bayes as a Language Model continued	CO3

Application Areas:

- Sentiment analysis in social media and reviews.

- Spam detection in email systems.
- Text classification for topic modeling.

Review Questions / Questions Appeared in the Previous Years (CO):

1	What are the key assumptions behind Naive Bayes classifiers, and how do they impact text classification? (CO3)
2	What steps are involved in training a Naive Bayes classifier for text classification? (CO3)
3	Can you walk through a step-by-step example of using Naive Bayes for spam email classification? (CO3)
4	How can Naive Bayes be optimized for sentiment analysis tasks, such as classifying positive and negative reviews? (CO3)
5	What are the common challenges in using Naive Bayes for sentiment analysis, and how can they be addressed? (CO3)
6	How does Naive Bayes compare to deep learning approaches for text classification in terms of accuracy and computational efficiency? (CO3)
7	How is Naive Bayes applied as a language model, and what are its limitations in this context? (CO3)
8	How does Naive Bayes compare to n-gram language models in text generation tasks? (CO3)
9	Assume the following likelihoods for each word being part of a positive or negative movie review, and equal prior probabilities for each class.

	pos	neg
I	0.09	0.16
always	0.07	0.06
like	0.29	0.06
foreign	0.04	0.15
films	0.08	0.11

What class will Naive bayes assign to the sentence “I always like foreign films.”? (CO3)

MODULE – IV

Title:	Information Retrieval and Lexical Resources	Appr. Time:	8 Hrs
MO:			RBT
At the end of the Module, the student will be able to:			
1. Design Features of Information Retrieval Systems			L3
2. Understand various IR models (classical, non-classical, cluster, fuzzy, and LSTM-based models).			L2
3. Explain Lexical Resources: WordNet, FrameNet. Utilize lexical resources such as WordNet, FrameNet, and POS taggers.			L2
4. Work with research corpora and understand their applications in NLP			L2

Lesson Schedule:

Lecture No.	Portion to be covered	CO
1	Information Retrieval: Design Features of Information Retrieval Systems	CO4
2	Information Retrieval Models – Classical, Non-classical	CO4
3	Alternative Models of Information Retrieval – Custer model	CO4
4	Fuzzy model	CO4
5	LSTM model, Major Issues in Information Retrieval	CO4

6	Lexical Resources: WordNet, FrameNet	CO4
7	Stemmers	CO4
8	Parts-of-Speech Tagger, Research Corpora	CO4

Application Areas:

- Search engines and query optimization.
- Lexical databases for semantic search (e.g., WordNet).
- Clustering models for document organization.

Review Questions / Questions Appeared in the Previous Years (CO):

1	Explain the Cluster and Fuzzy models of IR system. (CO4)
2	Explain WordNet with its applications. (CO4)
3	Describe the following: • Custer Modeling • Fuzzy Modeling • Stemming (CO4)
4	Explain Boolean and vector space information retrieval models. (CO4)
5	Explain basic Information Retrieval process with a neat diagram. (CO4)
6	Explain Classical, Non-classical retrieval model. (CO4)
7	Explain LSTM model. (CO4)
8	Explain, how stemming of a word is done? (CO4)

MODULE – V

Title:	Machine Translation	Appr. Time:	8 Hrs
MO:			RBT
At the end of the Module, the student will be able to:			
1. Describe language divergences and typology in machine translation.			L2
2, Implement machine translation using encoder-decoder models.			L3
3. Address challenges in low-resource language translation.			L2
4. Evaluate machine translation models while considering bias and ethical issues			L2

Lesson Schedule:

Lecture No.	Portion to be covered	CO
1	Introduction to Machine Translation	CO5
2	Language Divergences	CO5
3	Typology	CO5
4	Machine Translation using Encoder-Decoder	CO5
5	Details of the Encoder-Decoder Model	CO5
6	Translating in Low-Resource Situations	CO5
7	MT Evaluation	CO5
8	Bias and Ethical Issues	CO5

Application Areas:

- Language translation tools (e.g., Google Translate).
- Cross-lingual document understanding.
- Machine-assisted content localization.

Review Questions / Questions Appeared in the Previous Years (CO):

1	What are some examples of syntactic divergences that pose challenges for translation models?

	(CO5)
2	How does the encoder-decoder architecture handle long sentences in translation? (CO5)
3	What are the main advantages and limitations of encoder-decoder models compared to traditional rule-based translation systems? (CO5)
4	How do different architectures like LSTMs, GRUs, and Transformers compare in machine translation tasks? (CO5)
5	What are the key metrics used for evaluating machine translation performance? (CO5)
6	How does bias in training data affect the fairness of Machine Translation Systems? (CO5)
7	How do ethical considerations influence the deployment of MT systems in real-world applications? (CO5)
8	How can transfer learning help in low-resource Machine Translation? (CO5)

F. LABORATORY CONTENTS:

Expt No.	Title of the Experiments	RBT	CO
1	Write a Python program for the following preprocessing of text in NLP: • Tokenization • Filtration • Script Validation • Stop Word Removal • Stemming	L3	CO1
2	Demonstrate the N-gram modeling to analyze and establish the probability distribution across sentences and explore the utilization of unigrams, bigrams, and trigrams in diverse English sentences to illustrate the impact of varying n-gram orders on the calculated probabilities.	L3	CO1
3	Investigate the Minimum Edit Distance (MED) algorithm and its application in string comparison and the goal is to understand how the algorithm efficiently computes the minimum number of edit operations required to transform one string into another. • Test the algorithm on strings with different type of variations (e.g., typos, substitutions, insertions, deletions) • Evaluate its adaptability to different types of input variations	L3	CO2
4	Write a program to implement top-down and bottom-up parser using appropriate context free grammar.	L3	CO2
5	Given the following short movie reviews, each labeled with a genre, either comedy or action: • fun, couple, love, love comedy • fast, furious, shoot action • couple, fly, fast, fun, fun comedy • furious, shoot, shoot, fun action • fly, fast, shoot, love action and A new document D: fast, couple, shoot, fly Compute the most likely class for D. Assume a Naive Bayes classifier and use add-1 smoothing for the likelihoods.	L3	CO3
6	Demonstrate the following using appropriate programming tool which illustrates the use of information retrieval in NLP:	L3	CO4

	• Study the various Corpus – Brown, Inaugural, Reuters, udhr with various methods like filelds, raw, words, sents, categories. • Create and use your own corpora (plaintext, categorical). • Study Conditional frequency distributions. • Study of tagged corpora with methods like tagged_sents, tagged_words. • Write a program to find the most frequent noun tags • Map Words to Properties Using Python Dictionaries • Study Rule based tagger, Unigram Tagger Find different words from a given plain text without any space by comparing this text with a given corpus of words. Also find the score of words.		
7	Write a Python program to find synonyms and antonyms of the word "active" using WordNet.	L3	CO4
8	Implement the machine translation application of NLP where it needs to train a machine translation model for a language with limited parallel corpora. Investigate and incorporate techniques to improve performance in low-resource scenarios.	L3	CO5
9	Open ended experiment – 1		
10	Open ended experiment – 2		

EXPERIMENTS

1. EXPERIMENT NO: 01

2. TITLE: **WRITE A PYTHON PROGRAM FOR THE FOLLOWING PREPROCESSING OF TEXT IN NLP: TOKENIZATION, FILTRATION, SCRIPT VALIDATION, STOP WORD REMOVAL, STEMMING**

3. LEARNING OBJECTIVES:
- Understand the fundamental concepts of text preprocessing in Natural Language Processing (NLP).
- Implement tokenization to break text into meaningful units.
- Apply filtration techniques to remove unwanted characters or symbols.
- Perform script validation to ensure text consistency.
- Remove stop words to retain only relevant information.
- Apply stemming techniques to reduce words to their root forms.
- Develop a Python program to automate the text preprocessing steps.

4. AIM:
- To develop a python program that performs text preprocessing techniques in natural language processing (NLP).

5. MATERIAL / EQUIPMENT REQUIRED:
- Python programming

6. THEORY / HYPOTHESIS:

Text preprocessing is a crucial step in Natural Language Processing (NLP) to prepare raw text for analysis. The key techniques include:

1. Tokenization – Splitting text into smaller units (tokens), such as words or sentences, for easier analysis.
2. Filtration – Removing unwanted characters, punctuation, and special symbols to clean the text.
3. Script Validation – Ensuring that the text is in the desired script (e.g., English) using language detection techniques.
4. Stop Word Removal – Eliminating commonly used words (e.g., "is," "the," "and") that do

not contribute to meaning.

5. Stemming – Reducing words to their root form (e.g., "running" → "run") to normalize text

7. PROCEDURE / PROGRAMME / ACTIVITY:

```
!pip install nltk
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
import re

# Download necessary resources
nltk.download('punkt_tab')
nltk.download('stopwords')

def preprocess_text(text):
    # Tokenization
    tokens = word_tokenize(text)
    print("Tokenized Words:", tokens)

    # Filtration (Removing punctuations and special characters)
    filtered_tokens = [word for word in tokens if word.isalnum()]
    print("Filtered Tokens:", filtered_tokens)

    # Script Validation (Removing non-English words/characters)
    validated_tokens = [word for word in filtered_tokens if re.match("^[A-Za-z]+$", word)]
    print("Script Validated Tokens:", validated_tokens)

    # Stop Word Removal
    stop_words = set(stopwords.words('english'))
    tokens_without_stopwords = [word for word in filtered_tokens if word.lower() not in
stop_words]
    print("Tokens after Stop Word Removal:", tokens_without_stopwords)

    # Stemming
    stemmer = PorterStemmer()
    stemmed_tokens = [stemmer.stem(word) for word in tokens_without_stopwords]
    print("Stemmed Tokens:", stemmed_tokens)
    return stemmed_tokens

# Example usage
text = "Natural Language Processing (NLP) is a fascinating field of Artificial Intelligence!"
preprocessed_text = preprocess_text(text)

# for script validation
!pip install langdetect
from langdetect import detect
text = "नमस्ते, आप कैसे हैं?"
lang = detect(text)
```

print(f"The language of the text is: {lang}")

8. RESULTS & CONCLUSIONS:

- **Tokenized Words:** ['Natural', 'Language', 'Processing', '(', 'NLP', ')', 'is', 'a', 'fascinating', 'field', 'of', 'Artificial', 'Intelligence', '!']
- **Filtered Tokens:** ['Natural', 'Language', 'Processing', 'NLP', 'is', 'a', 'fascinating', 'field', 'of', 'Artificial', 'Intelligence']
- **Script Validated Tokens:** ['Natural', 'Language', 'Processing', 'NLP', 'is', 'a', 'fascinating', 'field', 'of', 'Artificial', 'Intelligence']
- **Tokens after Stop Word Removal:** ['Natural', 'Language', 'Processing', 'NLP', 'fascinating', 'field', 'Artificial', 'Intelligence']
- **Stemmed Tokens:** ['natur', 'languag', 'process', 'nlp', 'fascin', 'field', 'artifici', 'intellig']
- **Script validation:** The language of the text is: hi

9. LEARNING OUTCOMES :

- Knowledge of fundamental text preprocessing techniques in NLP.
- Enhance the quality of text data for NLP applications such as sentiment analysis and text classification.

10. APPLICATION AREAS:

- Sentiment Analysis
- Text Classification
- Plagiarism Detection
- Information Retrieval.
- Machine Translation

11. REMARKS:

-

1. EXPERIMENT NO: 02

2. TITLE: **N-GRAM MODELING FOR PROBABILITY DISTRIBUTION AND ANALYSIS OF UNIGRAMS, BIGRAMS, AND TRIGRAMS IN ENGLISH SENTENCES**

3. LEARNING OBJECTIVES:

- Understand the concept of N-gram modeling in Natural Language Processing (NLP).
- Analyze probability distribution across sentences using N-gram models.
- Explore the impact of different N-gram orders (unigrams, bigrams, trigrams) on text analysis.

4. AIM:

To demonstrate N-gram modeling for analyzing and establishing probability distribution across sentences and explore the impact of unigrams, bigrams, and trigrams on probability calculations.

5. MATERIAL / EQUIPMENT REQUIRED:

- Python Programming

6. THEORY / HYPOTHESIS:

N-gram modeling is a statistical language modeling technique used in NLP to predict the likelihood of a word sequence. It works by analyzing adjacent words (N-grams) in a sentence:

1. Unigram Model (N=1) – Considers individual words without context.
2. Bigram Model (N=2) – Considers word pairs, improving contextual understanding.

3. Trigram Model (N=3) – Considers three-word sequences, further refining predictions.

The probability of a sentence is estimated using conditional probabilities based on N-gram occurrences. The formula for a bigram model is:

$$P(W_n|W_{n-1}) = \frac{C(W_{n-1}, W_n)}{C(W_{n-1})}$$

where $C(W_{n-1}, W_n)$ is the count of the bigram, and $C(W_{n-1})$ is the count of the previous word.

Higher-order N-grams capture more context but require larger datasets for accurate probabilities.

7. PROCEDURE / PROGRAMME / ACTIVITY:

```
!pip install nltk
import nltk
from nltk import word_tokenize, ngrams
from collections import Counter
import math
nltk.download('punkt_tab')

def generate_ngrams(text, n): #Generate n-grams from the given text.
    tokens = word_tokenize(text.lower()) #Tokenization and lowercase conversion
    return list(ngrams(tokens, n))
def calculate_ngram_probabilities(ngrams_list): #Calculate probability distribution for the given n-grams
    ngram_counts = Counter(ngrams_list) # Count occurrences of each n-gram
    total_ngrams = sum(ngram_counts.values()) #Total occurrences
    probabilities = {ngram: count / total_ngrams for ngram, count in
            ngram_counts.items()}
    return probabilities

sentences = [ "I love programming.",
              "Programming is fun.",
              "I love coding and programming."]
text = " ".join(sentences) #Combine all sentences into one text (our training corpus) or directly above text can be used.
# Unigrams, Bigrams, and Trigrams
unigrams = generate_ngrams(text, 1)
bigrams = generate_ngrams(text, 2)
trigrams = generate_ngrams(text, 3)
# Probability distribution for each n-gram order
unigram_prob = calculate_ngram_probabilities(unigrams)
bigram_prob = calculate_ngram_probabilities(bigrams)
trigram_prob = calculate_ngram_probabilities(trigrams)
# Displaying Results
for unigram, prob in unigram_prob.items():
    print("\nUnigram Probabilities:")
    print(f"{unigram}: {prob:.4f}")
for bigram, prob in bigram_prob.items():
```

```
    print("\nBigram Probabilities:")
    print(f"{bigram}: {prob:.4f}")
for trigram, prob in trigram_prob.items():
    print("\nTrigram Probabilities:")
    print(f"{trigram}: {prob:.4f}")
```

For a given test sequence, total probability calculation with respect to corpus.

```
#Test sentence
test_sentence = "I love programming"

def compute_total_probability(test_sentence,  bigram_prob, trigram_prob):
    #Tokenize the test sentence
    test_words = nltk.word_tokenize(test_sentence.lower())
    # Compute bigram probability (P(w2 | w1) * P(w3 | w2) * ...)
    bigram = 1
    for i in range(1, len(test_words)):
        bigrams = (test_words[i-1], test_words[i])
        if bigrams in bigram_prob:
            bigram *= bigram_prob[bigrams]
        else:
            bigram *= 1e-10  #If bigram is not in bigram model, assign a very low probability
(smoothing)
    #Compute trigram probability (P(w3 | w1, w2) * P(w4 | w2, w3) * ...)
    trigram = 1
    for i in range(2, len(test_words)):
        trigrams = (test_words[i-2], test_words[i-1], test_words[i])
        if trigrams in trigram_prob:
            trigram *= trigram_prob[trigrams]
        else:
            trigram *= 1e-10  # If trigram is not in trigram model, assign a very low probability
(smoothing)
    return bigram, trigram
# Compute total probabilities for the test sentence
bigram_prob, trigram_prob = compute_total_probability(test_sentence,  bigram_prob,
trigram_prob)

# Print the results
print(f"Total Probability (Bigram) for '{test_sentence}': {bigram_prob:.10f}")
print(f"Total Probability (Trigram) for '{test_sentence}': {trigram_prob:.10f}")
```

8. RESULTS & CONCLUSIONS:

Unigram Probabilities:
('i',): 0.1429
('love',): 0.1429
('programming',): 0.2143
('.',): 0.2143
('is',): 0.0714

('fun',): 0.0714
('coding',): 0.0714
('and',): 0.0714
Bigram Probabilities:
('i', 'love'): 0.1538
('love', 'programming'): 0.0769
('programming', '.'): 0.1538
('.', 'programming'): 0.0769
('programming', 'is'): 0.0769
('is', 'fun'): 0.0769
('fun', '.'): 0.0769
('.', 'i'): 0.0769
('love', 'coding'): 0.0769
('coding', 'and'): 0.0769
('and', 'programming'): 0.0769
Trigram Probabilities:
('i', 'love', 'programming'): 0.0833
('love', 'programming', '.'): 0.0833
('programming', '.', 'programming'): 0.0833
('.', 'programming', 'is'): 0.0833
('programming', 'is', 'fun'): 0.0833
('is', 'fun', '.'): 0.0833
('fun', '.', 'i'): 0.0833
('.', 'i', 'love'): 0.0833
('i', 'love', 'coding'): 0.0833
('love', 'coding', 'and'): 0.0833
('coding', 'and', 'programming'): 0.0833
('and', 'programming', '.'): 0.0833

Total Probability (Bigram) for 'I love programming': 0.0118343195
Total Probability (Trigram) for 'I love programming': 0.0833333333

9. LEARNING OUTCOMES :

- Understand and apply the concept of N-gram modeling in NLP.
- Analyze probability distribution across sentences using unigrams, bigrams, and trigrams.
- Compute the probability of a sentence using N-gram frequency counts

10. APPLICATION AREAS:

- Text Prediction
- Speech Recognition
- Machine Translation
- Sentiment Analysis
- Spam Detection

11. REMARKS:

-

COURSE PLAN

1. EXPERIMENT NO: 03

2. TITLE: **INVESTIGATION OF THE MINIMUM EDIT DISTANCE (MED) ALGORITHM AND ITS APPLICATION IN STRING COMPARISON**

3. LEARNING OBJECTIVES:

- Understand the concept of the Minimum Edit Distance algorithm and its significance in string comparison.
- Implement the MED algorithm using dynamic programming to compute the edit distance between two strings.
- Analyze the impact of different types of variations (insertions, deletions, substitutions) on the computed edit distance.
- Evaluate the adaptability of the algorithm to different input variations and assess its real-world applications in areas like spell checking and text similarity analysis.
- Compare the MED algorithm's efficiency with alternative string comparison techniques. To understand the working of star topology

4. AIM:

Implement Minimum Edit Distance (MED) algorithm, understand its efficiency in computing the minimum number of edit operations required to transform one string into another, and evaluate its adaptability to various types of string variations.

5. MATERIAL / EQUIPMENT REQUIRED:

- Python programming

6. THEORY / HYPOTHESIS:

The Minimum Edit Distance (MED) algorithm, also known as the Levenshtein Distance, using dynamic programming. It efficiently computes the minimum number of edit operations (insertion, deletion, substitution) required to transform one string into another. The possible operations are:

1. Insertion: Adding a character (e.g., "kitten" → "kittene")
2. Deletion: Removing a character (e.g., "kitten" → "kiten")
3. Substitution: Replacing one character with another (e.g., "kitten" → "sitten")

MED Algorithm Handles typos effectively by detecting insertions, deletions, and substitutions. MED Algorithm useful in spell checkers, DNA sequence alignment, and NLP (e.g., autocorrect systems). It is used in Computational complexity is O(m × n), making it feasible for moderate-length strings.

7. PROCEDURE / PROGRAMME / ACTIVITY:

```
def min_edit_distance(str1, str2):
   m, n = len(str1), len(str2)
# Initialize a matrix to store distances
   dp = [[0 for _ in range(n+1)] for _ in range(m+1)]
# Base cases: Transforming empty string to another string
   for i in range(m+1):
    dp[i][0] = i     #Cost of deleting all characters
    for j in range(n+1):
     dp[0][j] = j    #Cost of inserting all characters
# Compute distances
     for i in range(1, m+1):
      for j in range(1, n+1):
       if str1[i-1] == str2[j-1]:     # No change needed if characters match
        dp[i][j] = dp[i-1][j-1]
```

```
        else:
          dp[i][j] = 1 + min(dp[i-1][j],  # Deletion
          dp[i][j-1],      # Insertion
          dp[i-1][j-1])     # Substitution
 return dp[m][n]
#Test the algorithm with different variations
test_cases = [("kitten", "sitting"),     # Substitutions, insertions
("flaw", "lawn"),     # Substitutions, insertions, deletions
("intention", "execution"),     # Multiple operations
("apple", "aple"),     # Deletion
("recieve", "receive"), ]    # Substitution
# Evaluate MED for each test case
for str1, str2 in test_cases:
  print(f"MED between '{str1}' and '{str2}': {min_edit_distance(str1, str2)}")
```

8. RESULTS & CONCLUSIONS:

MED between 'kitten' and 'sitting': 3
MED between 'flaw' and 'lawn': 2
MED between 'intention' and 'execution': 5
MED between 'apple' and 'aple': 1
MED between 'recieve' and 'receive': 2

9. LEARNING OUTCOMES :

- Demonstrate how the Minimum Edit Distance algorithm works through coding and practical experiments.
- Apply the algorithm to different string variations (e.g., typos, misspellings, word transformations).
- Interpret the edit distance results and their implications in real-world applications like spell correction, DNA sequence alignment, and natural language processing.

10. APPLICATION AREAS:

- Spell checking (e.g., Google auto-correct)
- Plagiarism detection
- DNA sequence alignment in bioinformatics
- Natural Language Processing for text similarity

11. REMARKS:

-

1. EXPERIMENT NO: 04

2. TITLE: **IMPLEMENTATION OF TOP-DOWN AND BOTTOM-UP PARSERS USING CONTEXT-FREE GRAMMAR**

3. LEARNING OBJECTIVES:

- Understand the concepts of Top-Down and Bottom-Up parsing techniques.
- Implement a Top-Down Parser using Recursive Descent.
- Implement a Bottom-Up Parser using Shift-Reduce parsing.
- Explore the role of Context-Free Grammar (CFG) in syntax analysis.

- Compare and contrast Top-Down and Bottom-Up parsing approaches.

4. AIM:
Write a program that implements both top-down and bottom-up parsers using appropriate context-free grammar (CFG).

5. MATERIAL / EQUIPMENT REQUIRED:
- Python Programming

6. THEORY / HYPOTHESIS:
Parsing is the process of analyzing a sequence of tokens to determine its grammatical structure based on a given formal grammar. There are two primary approaches to parsing:

1. Top-Down Parsing:
   - Starts from the start symbol and attempts to derive the input string by applying production rules.
   - A common method is Recursive Descent Parsing, where each non-terminal in the grammar is implemented as a function that tries to match input tokens.
   - Example: Predictive parsing without backtracking.
2. Bottom-Up Parsing:
   - Begins with input tokens and attempts to reconstruct the derivation sequence by reducing the input to the start symbol.
   - Shift-Reduce Parsing is a well-known bottom-up parsing technique.
   - Used in tools like YACC and Bison for compiler design.

- Context-Free Grammar (CFG): A CFG consists of:
  - A set of non-terminals (e.g., S, A, B)
  - A set of terminals (e.g., a, b, c)
  - A set of production rules (e.g., S -> AB, A -> a)
  - A start symbol (e.g., S)
- Example CFG:
  - S -> aAB
  - A -> b | ε
  - B -> c

7. PROCEDURE / PROGRAMME / ACTIVITY:

Top-Down Parsing:

```
class TopDownParser:
 def __init__(self, input_string):
  self.input = input_string
  self.index = 0
 def match(self, char):
  if self.index < len(self.input) and self.input[self.index] == char:
   self.index += 1
   return True
  return False
 def S(self):
  if self.match('a') and self.A() and self.B():
   return True
  return False
 def A(self):
  if self.match('b'):
```

```
    return True
  return True # Epsilon transition
 def B(self):
  return self.match('c')
 def parse(self):
  return self.S() and self.index == len(self.input)
# Example Usage
input_string = "abc"
td_parser = TopDownParser(input_string)
print("Top-Down Parsing Result:", td_parser.parse())
```

Bottom-Up Parsing:

```
class BottomUpParser:
 def __init__(self, input_string):
  self.input = list(input_string)
  self.stack = []
 def shift(self):
  if self.input:
    self.stack.append(self.input.pop(0))
 def reduce(self):
  if ''.join(self.stack[-2:]) == 'bc':
    self.stack[-2:] = ['A']
  elif ''.join(self.stack[-2:]) == 'aA':
    self.stack[-2:] = ['S']
  elif ''.join(self.stack) == 'S':
    return True
  return False
 def parse(self):
  while self.input or len(self.stack) > 1:
    self.shift()
    while self.reduce():
      pass
  return self.stack == ['S']
# Example Usage
input_string = "abc"
bu_parser = BottomUpParser(input_string)
print("Bottom-Up Parsing Result:", bu_parser.parse())
```

8. RESULTS & CONCLUSIONS:

Top-Down Parsing Result: True
Bottom-Up Parsing Result: True

9. LEARNING OUTCOMES :

- Knowledge of parsing techniques used in compilers.
- Construct a CFG suitable for parsing simple expressions or language constructs.
- Compare the efficiency and limitations of Top-Down and Bottom-Up parsing methods

10. APPLICATION AREAS:

- Natural Language Sentence Generation
- Simple Grammar Checkers

- Chart Parsing
- Dependency Parsing

11. REMARKS:

-

1. EXPERIMENT NO: 05

2. TITLE: **TEXT CLASSIFICATION USING NAÏVE BAYES CLASSIFIER WITH ADD-1 SMOOTHING**

3. LEARNING OBJECTIVES:
- To Understand the principles of the Naïve Bayes classifier for text classification.
- Explore the concept of Add-1 Smoothing to handle zero probabilities.

4. AIM:
To write a program to classify a given document into the most likely genre (Comedy or Action) using a Naïve Bayes classifier with Add-1 smoothing, based on previously labeled movie reviews.

5. MATERIAL / EQUIPMENT REQUIRED:
- Python Programming

6. THEORY / HYPOTHESIS:

The **Naïve Bayes classifier** is a probabilistic machine learning model based on **Bayes' Theorem**, which assumes that features (words) in a document are conditionally independent given the class. The classifier assigns a new document **D** to the class **C** that maximizes the posterior probability:

$$P(C|D) \propto P(C) \prod P(w_i|C)$$

To avoid zero probability issues, Laplace Smoothing (Add-1 Smoothing) is applied:

$$\hat{P}(w_i|c) = \frac{count(w_i, c) + 1}{\sum_{w \in V} (count(w, c) + 1)} = \frac{count(w_i, c) + 1}{\left(\sum_{w \in V} count(w, c)\right) + |V|}$$

Where: |V| is the total number of unique words in the vocabulary. Using this method, the class with the highest posterior probability is selected as the predicted category for the document.

7. PROCEDURE / PROGRAMME / ACTIVITY:

```
from collections import defaultdict
import math
# Training dataset with (text, label)
data = [
    ("fun, couple, love, love, comedy", "Comedy"),
    ("fast, furious, shoot, action", "Action"),
    ("couple, fly, fast, fun, fun, comedy", "Comedy"),
    ("furious, shoot, shoot, fun, action", "Action"),
    ("fly, fast, shoot, love, action", "Action")
]
test_doc = "fast, couple, shoot, fly"
# Tokenization function
def tokenize(text):
    return text.lower().split(", ")
```

```
# Naïve Bayes Training Function
def train_naive_bayes(data):
    word_counts = defaultdict(lambda: defaultdict(int))  # Word frequency per class
    class_counts = defaultdict(int)  # Document count per class
    vocab = set()  # Unique words in training data
    for text, label in data:
        class_counts[label] += 1
        for word in tokenize(text):
            word_counts[label][word] += 1
            vocab.add(word)
    return word_counts, class_counts, len(vocab)
# Naïve Bayes Classification Function
def classify(text, word_counts, class_counts, vocab_size):
    priors = {c: math.log(class_counts[c] / sum(class_counts.values())) for c in class_counts}
    posteriors = {c: priors[c] + sum( math.log((word_counts[c][w] + 1) /
(sum(word_counts[c].values()) + vocab_size)) # Laplace smoothing
                                for w in tokenize(text)) for c in class_counts}
    return max(posteriors, key=posteriors.get)
# Train and test
word_counts, class_counts, vocab_size = train_naive_bayes(data)
predicted_class = classify(test_doc, word_counts, class_counts, vocab_size)
print(f"Test Doc: {test_doc}")
print(f"Predicted Class: {predicted_class}")
```

8. RESULTS & CONCLUSIONS:

Test Doc: fast, couple, shoot, fly
Predicted Class: Action

9. LEARNING OUTCOMES :

- Apply the Naïve Bayes classifier to text classification problems.
- Implement probability-based decision making for text categorization.
- Understand the impact of smoothing techniques in handling unseen words.
- Accurately compute and interpret prior probabilities, likelihoods, and posterior probabilities.
- Develop an efficient approach to categorize text-based datasets in real-world applications

14. APPLICATION AREAS:

- Spam Detection
- Sentiment Analysis
- Document Classification
- Medical Diagnosis & Text Mining
- Fake News Detection

15. REMARKS:

-

COURSE PLAN

1. EXPERIMENT NO: 06

2. TITLE: **EXPLORING INFORMATION RETRIEVAL TECHNIQUES IN NLP USING PYTHON**

3. LEARNING OBJECTIVES:

- To understand the role of information retrieval in Natural Language Processing (NLP).
- To Explore different corpus datasets such as Brown, Inaugural, Reuters, and Universal Declaration of Human Rights (UDHR).
- To Learn various methods to process and analyze corpus data, including fields, raw text, words, sentences, and categories.
- To study and apply conditional frequency distributions in corpus analysis.
- To implement a method to segment a given text without spaces using a reference corpus and compute word scores.

4. AIM:

Develop a program to demonstrate and implement various information retrieval techniques in NLP using appropriate programming tools, focusing on corpus exploration, tagging, frequency analysis, and text segmentation.

5. MATERIAL / EQUIPMENT REQUIRED:

- Python Programming

6. THEORY / HYPOTHESIS:

Information Retrieval (IR) in NLP involves extracting relevant information from large text datasets and organizing it for effective use. This includes working with corpora, frequency distributions, tagging methods, and text segmentation techniques.

1. Corpora in NLP:
   - Corpora are structured collections of text used for linguistic analysis and training NLP models.
   - Examples:
     - Brown Corpus – First million-word electronic corpus of English.
     - Inaugural Corpus – Collection of US presidential speeches.
     - Reuters Corpus – News articles categorized by topic.
     - UDHR Corpus – Multilingual versions of the Universal Declaration of Human Rights.
2. Corpus Processing Techniques:
   - raw(), words(), sents(), categories() functions help retrieve different aspects of a corpus.
   - Custom corpora can be created and categorized based on application needs.
3. Conditional Frequency Distributions (CFD): Analyzing frequency patterns in different contexts, such as word usage across categories.
4. Tagged Corpora and POS Tagging:
   - POS tagging is assigning word categories (noun, verb, adjective, etc.) using techniques like:
     - Rule-based tagger (manually defined rules).
     - Unigram tagger (assigns most frequent tag for each word).
   - tagged_sents(), tagged_words() provide tagged versions of corpora.
5. Finding Most Frequent Noun Tags: Extracting and analyzing frequently used noun tags from tagged corpora.
6. Mapping Words to Properties Using Dictionaries: Using Python dictionaries to store and retrieve word-related attributes.
7. Text Segmentation & Scoring Words:
   - Breaking continuous text without spaces into meaningful words using corpus-based

comparison.
- Assigning scores based on frequency and context relevance.

7. PROCEDURE / PROGRAMME / ACTIVITY:

```
import nltk
from nltk.corpus import brown, inaugural, reuters, udhr
from nltk import FreqDist, ConditionalFreqDist
from nltk.tokenize import word_tokenize
from nltk.tag import pos_tag, UnigramTagger
from nltk.probability import FreqDist
from nltk.corpus import PlaintextCorpusReader
# Download required datasets
nltk.download('brown')
nltk.download('inaugural')
nltk.download('reuters')
nltk.download('udhr')
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
# 1. Study various Corpora
def explore_corpora():
  print("\nBrown Corpus Categories:", brown.categories())
  print("First 10 words in Inaugural Corpus:", inaugural.words()[:10])
  print("First 10 words in Reuters Corpus:", reuters.words()[:10])
  print("First 10 words in UDHR Corpus (English):", udhr.words('English-Latin1')[:10])
  return explore_corpora
explore_corpora()
# 2. Create and use a custom corpus
def create_custom_corpus():
  corpus_root = "path/to/your/folder"
  wordlists = PlaintextCorpusReader(corpus_root, '.*\.txt')
  print("Files in Corpus:", wordlists.fileids())
  print("First 20 words:", wordlists.words()[:20])
  return create_custom_corpus
create_custom_corpus()
# 3. Conditional Frequency Distribution
def conditional_frequency():
  cfd = ConditionalFreqDist(
      (genre, word.lower())
      for genre in brown.categories()
      for word in brown.words(categories=genre)
      )
  print("\nMost common words in 'news' category:", cfd['news'].most_common(10))
  return conditional_frequency
conditional_frequency()
# 4. Study Tagged Corpora & Find Most Frequent Noun Tags
def frequent_noun_tags():
  tagged_words = brown.tagged_words()
```

```
 noun_tags = [word for word, tag in tagged_words if tag.startswith('NN')]
 fdist = FreqDist(noun_tags)
 print("\nMost Frequent Nouns:", fdist.most_common(10))
 return frequent_noun_tags
frequent_noun_tags()
# 5. Map Words to Properties Using Python Dictionaries
def word_properties():
 word_dict = {"AI": "Artificial Intelligence", "NLP": "Natural Language Processing", "ML":
"Machine Learning"}
 print("\nWord Properties:", word_dict["NLP"])
 return word_properties
word_properties()
# 6. Rule-based Tagger & Unigram Tagger
def unigram_tagger_demo():
   tagged_sents = brown.tagged_sents(categories='news')
   train_data = tagged_sents[:4000]
   test_data = tagged_sents[4000:]
   unigram_tagger = UnigramTagger(train_data)
   accuracy = unigram_tagger.evaluate(test_data)
   print("\nUnigram Tagger Accuracy:", round(accuracy, 4))
   return unigram_tagger  # Return the actual tagger if needed
unigram_tagger_demo()
# 7. Word Segmentation from Continuous Text
def segment_text(text, corpus_words):
   words = []
   while text:
      for i in range(len(text), 0, -1):
         if text[:i] in corpus_words:
            words.append(text[:i])
            text = text[i:]
            break
      else:
         # If no match, take one character to avoid infinite loop
         words.append(text[0])
         text = text[1:]
   return words
def word_segmentation():
   corpus_words = set(w.lower() for w in brown.words())  # Lowercase for matching
   text = "thisisatextwithoutspaces"
   segmented_words = segment_text(text, corpus_words)
   print("\nSegmented Words:", segmented_words)
word_segmentation()
```

8. RESULTS & CONCLUSIONS:

1.Brown Corpus Categories: ['adventure', 'belles_lettres', 'editorial', 'fiction', 'government', 'hobbies', 'humor', 'learned', 'lore', 'mystery', 'news', 'religion', 'reviews', 'romance', 'science_fiction']

First 10 words in Inaugural Corpus: ['Fellow', '-', 'Citizens', 'of', 'the', 'Senate', 'and', 'of', 'the', 'House']

COURSE PLAN

First 10 words in Reuters Corpus: ['ASIAN', 'EXPORTERS', 'FEAR', 'DAMAGE', 'FROM', 'U', '.', 'S', '.-', 'JAPAN']
First 10 words in UDHR Corpus (English): ['Universal', 'Declaration', 'of', 'Human', 'Rights', 'Preamble', 'Whereas', 'recognition', 'of', 'the']
2.Files in Corpus: (Mention the result)
First 20 words:
3.Most common words in 'news' category: [('the', 6386), (',', 5188), ('.', 4030), ('of', 2861), ('and', 2186), ('to', 2144), ('a', 2130), ('in', 2020), ('for', 969), ('that', 829)]
4.Most Frequent Nouns: [('time', 1555), ('man', 1148), ('Af', 994), ('years', 942), ('way', 883), ('people', 809), ('men', 736), ('world', 684), ('life', 676), ('year', 647)]
5.Word Properties: Natural Language Processing
<ipython-input-36-dc43261bd35a>:55: DeprecationWarning:
Function evaluate() has been deprecated. Use accuracy(gold)
instead.
accuracy = unigram_tagger.evaluate(test_data)
6.Unigram Tagger Accuracy: 0.8111
7.Segmented Words: ['this', 'is', 'ate', 'x', 't', 'without', 'spaces']

9. LEARNING OUTCOMES :

- Knowledge of different types of corpora in NLP.
- Create custom corpora and apply methods to process them.
- Implement and interpret conditional frequency distributions.
- Implement rule-based and unigram taggers for POS tagging.
- Knowledge of segmentation techniques to identify words in unstructured text.
- Evaluate word scores based on a given corpus.
- Knowledge of how to control congestion using Leaky bucket algorithm

10. APPLICATION AREAS:

- Text Processing & Preprocessing
- Information Retrieval Systems
- Speech & Language Processing
- Sentiment Analysis
- Machine Translation

11. REMARKS:

-

1. EXPERIMENT NO: 07

2. TITLE: **FINDING SYNONYMS AND ANTONYMS USING WORDNET IN PYTHON**

3. LEARNING OBJECTIVES:

- To understand how to interact with WordNet using Python and the nltk library.
- To learn how to find synonyms and antonyms of a word programmatically.
- To understand the concepts of synsets, lemmas, and antonyms in the context of natural language processing.
- To practice using Python libraries to access lexical resources for word analysis

COURSE PLAN

4. AIM:
Write a Python program that retrieves synonyms and antonyms for a given word ("active") using the WordNet lexical database, and to understand how to extract semantic relationships between words in natural language processing (NLP).

5. MATERIAL / EQUIPMENT REQUIRED:
- Python Programming

6. THEORY / HYPOTHESIS:
WordNet is a large lexical database of English, in which words are grouped into sets of synonyms called **synsets**. Each synset represents a concept, and words that share a meaning are connected. WordNet also provides antonyms for many words, which are words with opposite meanings.
- Synset: A group of synonymous words or phrases.
- Lemma: A canonical form of a word, used as a representative of a synset.
- Antonym: A word that expresses the opposite meaning of another word.

By utilizing nltk, a popular Python library for NLP, we can interact with WordNet to explore words' meanings, relationships, and contextual usage.

7. PROCEDURE / PROGRAMME / ACTIVITY:

```
import nltk
from nltk.corpus import wordnet
# Ensure that WordNet is downloaded
nltk.download('wordnet')
def get_synonyms_antonyms(word):
    synonyms = set()
    antonyms = set()
    for syn in wordnet.synsets(word):
        for lemma in syn.lemmas():
            synonyms.add(lemma.name())
            if lemma.antonyms():
                antonyms.add(lemma.antonyms()[0].name())
    return synonyms, antonyms
# Get synonyms and antonyms of the word "active"
word = "active"
synonyms, antonyms = get_synonyms_antonyms(word)
print(f" Word: {word}")
print(" Synonyms:", synonyms)
print(" Antonyms:", antonyms)
```

8. RESULTS & CONCLUSIONS:
Word: active
Synonyms: {'combat-ready', 'dynamic', 'active', 'fighting', 'alive', 'participating', 'active_voice', 'active_agent'}
Antonyms: {'dormant', 'passive_voice', 'stative', 'passive', 'quiet', 'extinct', 'inactive'}

9. LEARNING OUTCOMES :
- Understand the structure of WordNet and its application in NLP.
- Extract synonyms and antonyms of words programmatically using the nltk library.

- Write Python code that accesses WordNet, analyzes synsets, and handles lexical information.
- Recognize the importance of lexical databases in natural language processing for various tasks like sentiment analysis, word sense disambiguation, etc.

10. APPLICATION AREAS:
- Sentiment Analysis
- Text Classification
- Natural Language Understanding
- Language Resources

11. REMARKS:

-

1. EXPERIMENT NO: 08

2. TITLE: **MACHINE TRANSLATION IN LOW-RESOURCE SCENARIOS USING NLP**

3. LEARNING OBJECTIVES:
- To understand the fundamentals of machine translation and its challenges in low-resource languages.
- To explore different machine learning and deep learning techniques used for machine translation.
- To implement a machine translation model for a language with limited parallel corpora.
- To investigate methods to enhance translation quality in low-resource settings.

4. AIM:
Write a Python program to develop and improve a machine translation model for a low-resource language by leveraging NLP techniques and optimizing model performance with limited parallel corpora.

5. MATERIAL / EQUIPMENT REQUIRED:
- Python Programming

6. THEORY / HYPOTHESIS:
Machine translation (MT) is a subfield of Natural Language Processing (NLP) that focuses on automatically translating text from one language to another. Standard MT approaches include:
- Rule-Based Machine Translation (RBMT) – Uses linguistic rules and dictionaries.
- Statistical Machine Translation (SMT) – Uses probabilistic models based on bilingual text corpora.
- Neural Machine Translation (NMT) – Utilizes deep learning techniques such as sequence-tosequence models and transformers.

Low-resource languages present a challenge due to the lack of parallel corpora, making it difficult to train high-quality models. Common techniques to improve translation in low-resource settings include:
- Transfer Learning – Using pre-trained models from high-resource languages.
- Back-Translation – Generating synthetic parallel data by translating monolingual text.
- Data Augmentation – Expanding available corpora using techniques like word embeddings and paraphrasing.

- Few-Shot and Zero-Shot Learning – Adapting models with minimal training data.
- Transformer-Based Approaches (e.g., BERT, T5, mBART) – Leveraging multi-lingual pre-trained models for translation.

7. PROCEDURE / PROGRAMME / ACTIVITY:

```
!pip install datasets
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
# Load dataset
dataset = load_dataset("cfilt/iitb-english-hindi")
# Load model and tokenizer
tokenizer = AutoTokenizer.from_pretrained("Helsinki-NLP/opus-mt-en-hi")
model = AutoModelForSeq2SeqLM.from_pretrained("Helsinki-NLP/opus-mt-en-hi")
# Select an English sentence from validation set
article = dataset['validation'][2]['translation']['en']
# Tokenize the input
inputs = tokenizer(article, return_tensors="pt")
# Generate translation
translated_tokens = model.generate(**inputs, max_length=256)
# Decode and print translation
translated_text = tokenizer.decode(translated_tokens[0], skip_special_tokens=True)
print(translated_text)
```

8. RESULTS & CONCLUSIONS:

- Dataset Used: IIT Bombay English-Hindi Parallel Corpus
- English Sentence: This is an example sentence from the dataset.
- Original Hindi Translation (from dataset): यह डेटासेट से लिया गया एक उदाहरण वाक्य है।
- Model-Generated Hindi Translation: यह डेटासेट से लिया गया एक उदाहरण वाक्य है।

9. LEARNING OUTCOMES :

- Explain the challenges and solutions for machine translation in low-resource languages.
- Train and evaluate a machine translation model using limited parallel corpora.
- Apply transfer learning and data augmentation techniques to enhance translation quality.
- Optimize the model's performance through hyperparameter tuning and evaluation metrics.

10. APPLICATION AREAS:

- Educational and Linguistic Research.
- Cross-Language Communication
- AI-Assisted Translation Tools
- Content Localization

11. REMARKS:

COURSE PLAN

H. INTERNAL ASSESSMENT TEST MODEL QUESTION PAPER

Dept: AI	Sem / Div: VI	Course: NLP	Course Code: BAI601
Date:	Time: 90 Min.	Max Marks:	Elective: N

Note: Answer any 2 FULL questions.

QN		Questions	Marks	RBT	CO
		PART A			
1	a	What are the main challenges in NLP?	5	L1	CO1
	b	Define Natural Language Processing (NLP) and its applications. Explain Karaka theory.	10	L2	CO1
	c	Explain the Naive Bayes classifier and its application in NLP.	10	L3	CO3
2	a	What is the Paninian framework, and how is it applied in NLP?	5	L1	CO1
	b	What is an N-gram model? Explain Unigram and Bigram with examples.	10	L2	CO1
	c	What steps are involved in training a Naive Bayes classifier for text classification?	10	L3	CO3
		PART B			
3	a	Describe the role of morphological parsing in NLP.	5	L1	CO2
	b	What is CYK Parsing Algorithm with example and where is it used?	10	L2	CO2
	c	What are the key assumptions behind Naive Bayes classifiers and how do they impact text classification?	10	L3	CO3
4	a	Explain Finite-State Automata and its role in NLP.	5	L1	CO2
	b	Differentiate between top-down and bottom-up parsing.	10	L2	CO2
	c	Can you walk through a step-by-step example of using Naive Bayes for spam email classification?	10	L3	CO3

G. CONTINUOUS INTERNAL EVALUATION

Evaluation	Weightage in Marks	
IA Test – 1	50	
IA Test – 2	50	
Total IA	100 scaled down to 15 marks [Minimum marks 06]	
One Assignment	**5marks**	**10 [Minimum marks 04]**
Class Test/Quiz/Seminar	**5marks**	
TOTAL	15+10=25 marks [Minimum marks 10]	
Laboratory component	25 marks [Minimum marks 10] [Record-15M and Lab internals-50M scale down to 10M]	
Final CIE marks	50 marks [Minimum marks 20]	